

CastagNet

Un logiciel qui reconnaît automatiquement la qualité des châtaignes à partir de photos

Mise en Situation Professionnelle (MESPR) — MSc BIHAR / ESTIA

Auteur : **Mario Caballero** · Dépôt du code : github.com/ayala2a/castagnet

Données

Modèle

Mise en production

Comment j'ai travaillé. Je me suis aidé d'une intelligence artificielle (Claude) comme assistant : pour analyser les données, retrouver les bonnes méthodes, écrire le code et corriger les erreurs. Les choix, le raisonnement et la direction du projet restent les miens ; l'assistant m'a surtout fait gagner du temps.

1. Le problème à résoudre

Une coopérative corse (le GRPTMC) trie ses châtaignes sèches sur une machine automatique. La machine prend chaque fruit en photo pendant qu'il défile, puis doit décider s'il est **bon (Conforme)**, **mauvais (NON Conforme)**, d'une variété particulière à écarter (**PIETRA**), ou si l'emplacement est simplement **vide**. On m'a confié le projet pour l'améliorer sur trois points : la qualité des données, la fiabilité du logiciel de reconnaissance, et sa capacité à tourner sur la vraie machine.

La règle imposée par la coopérative est simple à comprendre et guide tout mon travail : **il vaut mieux jeter un bon fruit par erreur que d'en laisser passer un mauvais**. Laisser passer un mauvais fruit gâche la farine et fait perdre le label qualité ; jeter un bon fruit fait juste perdre un peu de marchandise. Concrètement, la coopérative demande que, parmi les fruits que le logiciel déclare « bons », **au moins 95 % le soient vraiment** (peu d'erreurs), et que le logiciel **retrouve au moins 85 % des bons fruits**. Le tout doit tourner en direct sur les 12 caméras de la machine, avec un ordinateur assez ancien.

Deux mots à retenir. On parlera souvent de deux mesures : la **précision** = « quand le logiciel dit bon, a-t-il raison ? » ; et le **rappel** = « retrouve-t-il bien tous les bons fruits ? ». La coopérative veut une précision très haute ($\geq 95\%$) et un bon rappel ($\geq 85\%$).

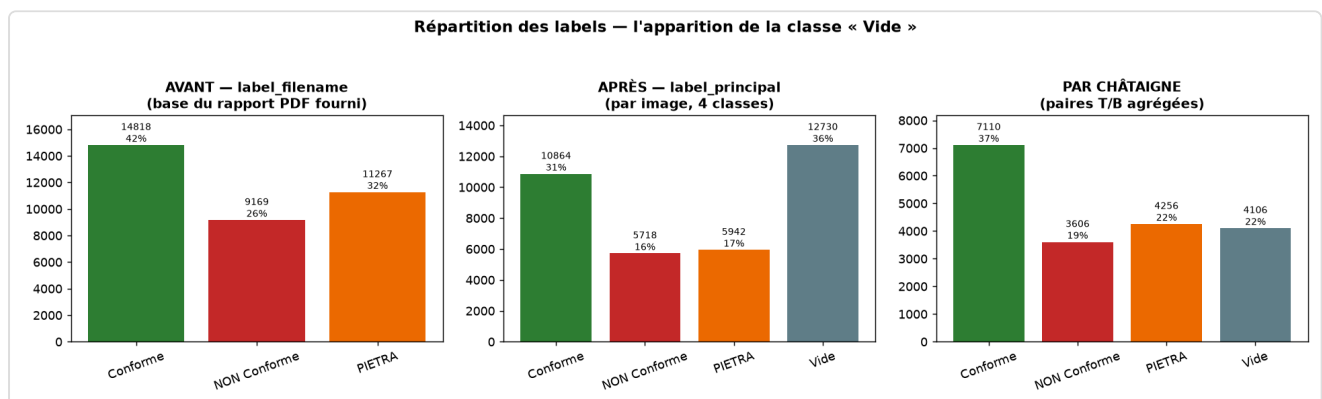
2. Ma ligne de conduite

Deux idées simples ont guidé tout le projet. La première : **une châtaigne, ce n'est pas une photo mais deux** — la machine la photographie du dessus ET du dessous. La seconde : **dans le doute, on écarte**. À partir de là, j'ai avancé dans l'ordre logique : d'abord bien comprendre et nettoyer les données, ensuite construire un logiciel qui décide à partir des deux photos, enfin vérifier qu'il tourne sur la vraie machine.

3. Le travail sur les données

3.1 Récupérer les données et les vérifier

J'ai d'abord récupéré les 35 254 photos fournies. En regardant de près, j'ai trouvé un problème important : le rapport qui accompagnait les données ne parlait que de trois catégories et oubliait complètement les emplacements **vides**. Or, en comparant l'étiquette d'origine (devinée à partir du nom du fichier) avec la vraie étiquette vérifiée par des humains, j'ai constaté que **36 % des photos étaient en réalité des emplacements vides**. Le rapport de départ était donc faux ; je l'ai refait correctement.

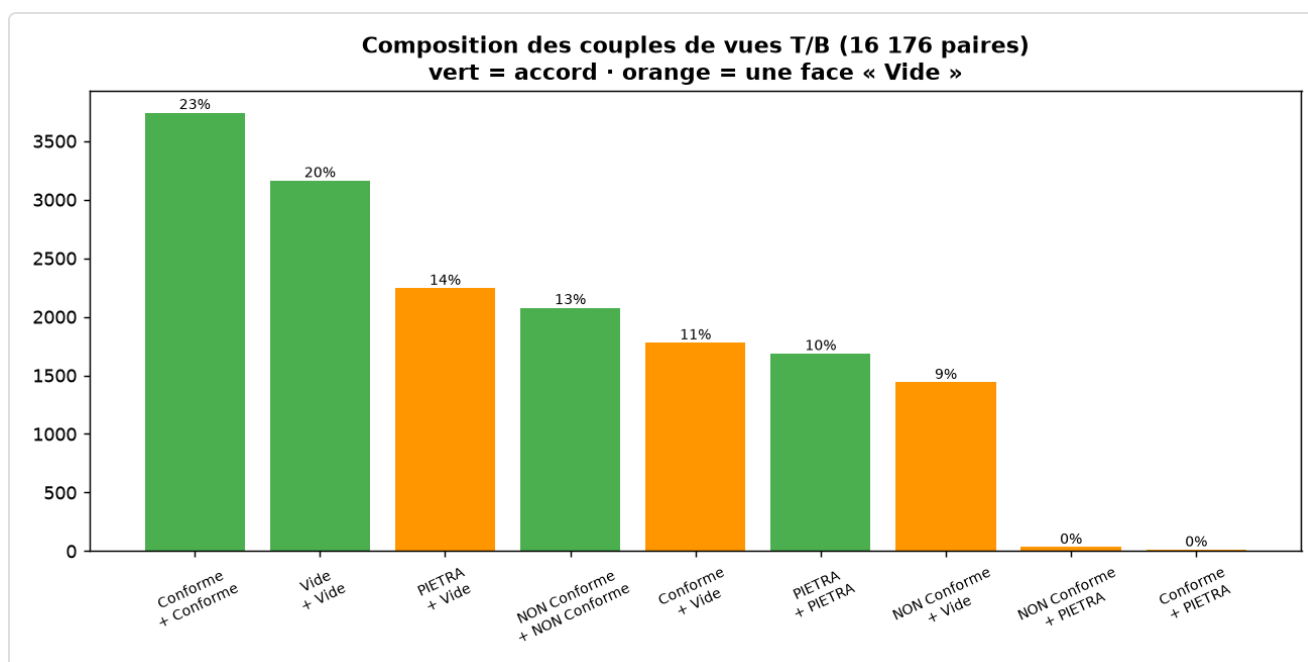


À gauche, l'ancien rapport (trois catégories) ; à droite, la réalité : les emplacements « vides » représentent plus d'un tiers des photos.

J'ai aussi terminé la vérification d'un lot de photos qui restait à contrôler, et j'ai remarqué que ce lot était très mal étiqueté au départ. Ça montre que les données n'étaient pas fiables telles quelles — une chose que le sujet demandait justement d'analyser.

3.2 Relier la photo du dessus et celle du dessous

Puisqu'un fruit a deux photos, il faut savoir lesquelles vont ensemble. Je les ai reliées grâce à leur nom de fichier. En vérifiant, j'ai découvert un point clé : quand les deux photos d'un même fruit ne sont pas d'accord sur la catégorie, **c'est toujours parce que l'une des deux ne voit pas le fruit** (il est hors du champ de cette caméra), jamais parce qu'elles voient deux qualités différentes.



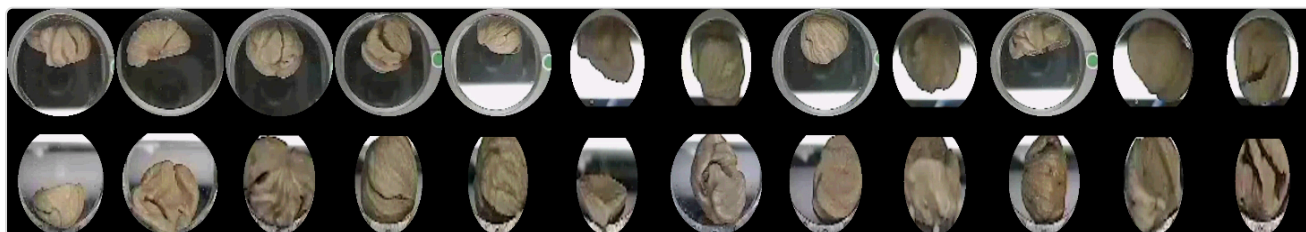
Les seuls désaccords entre les deux photos viennent d'une vue « vide » ; jamais d'un vrai conflit de qualité.

Pourquoi c'est important. Ça prouve qu'on ne peut pas se fier à une seule photo : environ un fruit sur trois semble « vide » sur l'une de ses deux photos alors qu'il est bien là. La bonne règle est donc simple : **si une photo voit le fruit, on lui fait confiance**, et en cas de vrai doute on choisit la catégorie la plus sévère — ce qui respecte la consigne « dans le doute, on écarte ».

3.3 Extraire et relier les images des vidéos

On m'a aussi donné deux vidéos de 30 secondes qui filment le même fruit, une du dessus et une du dessous, mais elles ne sont **pas parfaitement synchronisées** : l'une démarre un tout petit peu avant l'autre. Il fallait retrouver ce décalage pour relier les bonnes images entre elles.

Pourquoi ne pas le faire à l'œil : comparer deux vidéos de 750 images au jugé n'est ni fiable ni sérieux. Je l'ai donc mesuré. J'ai repéré, dans chaque vidéo, les moments où un fruit passe, et j'ai vu qu'ils passaient exactement au même rythme dans les deux — c'est donc bien le même flux. En comparant ces moments, j'obtiens un décalage **clair et unique de 5 images (0,2 seconde)**. J'ai vérifié une paire à l'écran (même fruit, même repère vert, vu sous deux angles), puis sorti une vingtaine de paires propres.



Fruits reliés depuis les vidéos : en haut la première vidéo, en bas la seconde ; chaque colonne est le même fruit vu des deux côtés.

3.4 Préparer les données pour l'apprentissage

Pour entraîner le logiciel, je raisonne par **fruit** (ses deux photos), pas par photo isolée. Je réparties les fruits en trois groupes : un pour apprendre, un pour régler, un pour tester. Point important : **les deux photos d'un même fruit restent toujours dans le même groupe**. Sinon, le logiciel pourrait « reconnaître » un fruit déjà vu et tricher sans le vouloir — il aurait de bonnes notes au test mais échouerait sur la vraie machine. J'ai vérifié que ce n'arrive jamais.

4. Le logiciel de reconnaissance

4.1 Les différentes versions, et pourquoi

J'ai construit plusieurs versions, de plus en plus fines, en me basant à chaque fois sur ce que les résultats me montraient :

- **Une version simple** qui ne regarde qu'une seule photo (la version de base demandée). Elle rate presque tous les fruits PIETRA — logique, le défaut ne se voit souvent que d'un côté.
- **Une version qui regarde les deux photos ensemble**. C'est celle qui colle au problème : deux photos, une décision. J'ai choisi un modèle de reconnaissance d'images **léger et rapide** (appelé MobileNet), parce que la vraie machine a un ordinateur modeste. Je n'ai pas utilisé de détection d'objet (type « encadrer le fruit ») : le fruit est déjà bien au centre, il suffit de le classer.
- **Une version qui regarde en plus la différence entre les deux photos**. En voyant que les résultats plafonnaient, je me suis dit : si un défaut n'est visible que d'un côté, alors les deux photos se ressemblent moins à cet endroit. J'ai donc donné au logiciel non seulement les deux photos, mais aussi **leur différence**. C'est ce qui a fait progresser le rappel.
- **Une version « radiale »**. Le sujet suggérait d'exploiter le fait que l'information est dans le disque central de l'image. J'ai donc **déroulé ce disque à plat** (comme si on l'ouvrait en partant du centre) avant de le montrer au logiciel. Avantage : peu importe comment le fruit est tourné, ça revient au même. C'est finalement ma **meilleure version**.

Une astuce utilisée partout. Pour fiabiliser les décisions, je fais analyser au logiciel plusieurs versions **pivotées** de la même photo, puis je prends la moyenne. Ça évite qu'un mauvais angle fausse le résultat.

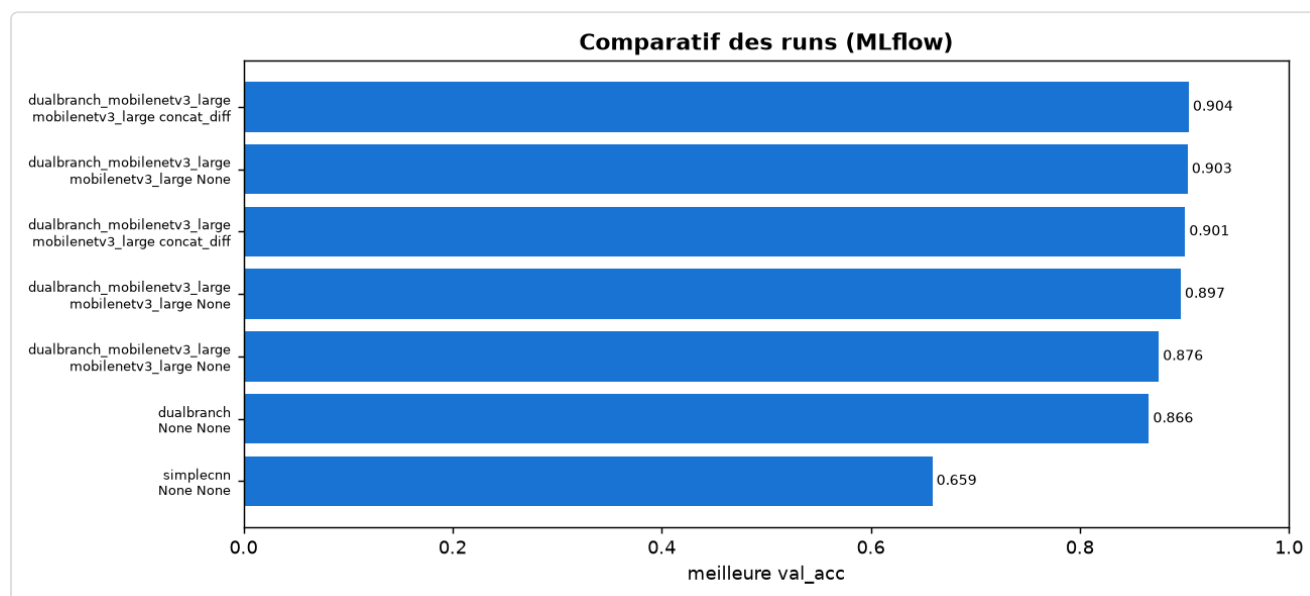
4.2 Comment le logiciel décide (et pourquoi comme ça)

Le logiciel ne se contente pas de choisir la catégorie la plus probable : ce serait risqué vu la consigne. À la place, il ne déclare un fruit « **bon** » que s'il en est **vraiment sûr** (au-dessus d'un niveau de confiance réglé exprès). C'est ce réglage qui garantit qu'au moins 95 % des fruits déclarés bons le sont réellement, tout en en retrouvant le plus possible. J'ai d'ailleurs choisi la meilleure version en visant directement cet objectif, pas une note globale.

4.3 Les résultats (sur des fruits jamais vus)

Version	Bonnes réponses (global)	Précision / rappel sur « bon »	Objectif atteint ?
Version simple (1 photo)	66 %	95 % / 30 %	non
Deux photos ensemble	86 %	96 % / 77 %	non (rappel)
+ la différence entre les photos	91 %	95 % / 88 %	oui
Version radiale (retenue)	92 %	95 % / 92 %	oui, avec de la marge

Le récit est clair : la version à une seule photo échoue (elle ne voit pas les défauts d'un côté) ; regarder les deux photos aide beaucoup ; ajouter leur différence fait passer l'objectif ; et la version radiale gagne encore. Chaque idée a été validée par les chiffres.



Comparaison des différentes versions testées (suivi automatique des essais).

4.4 Analyse des erreurs

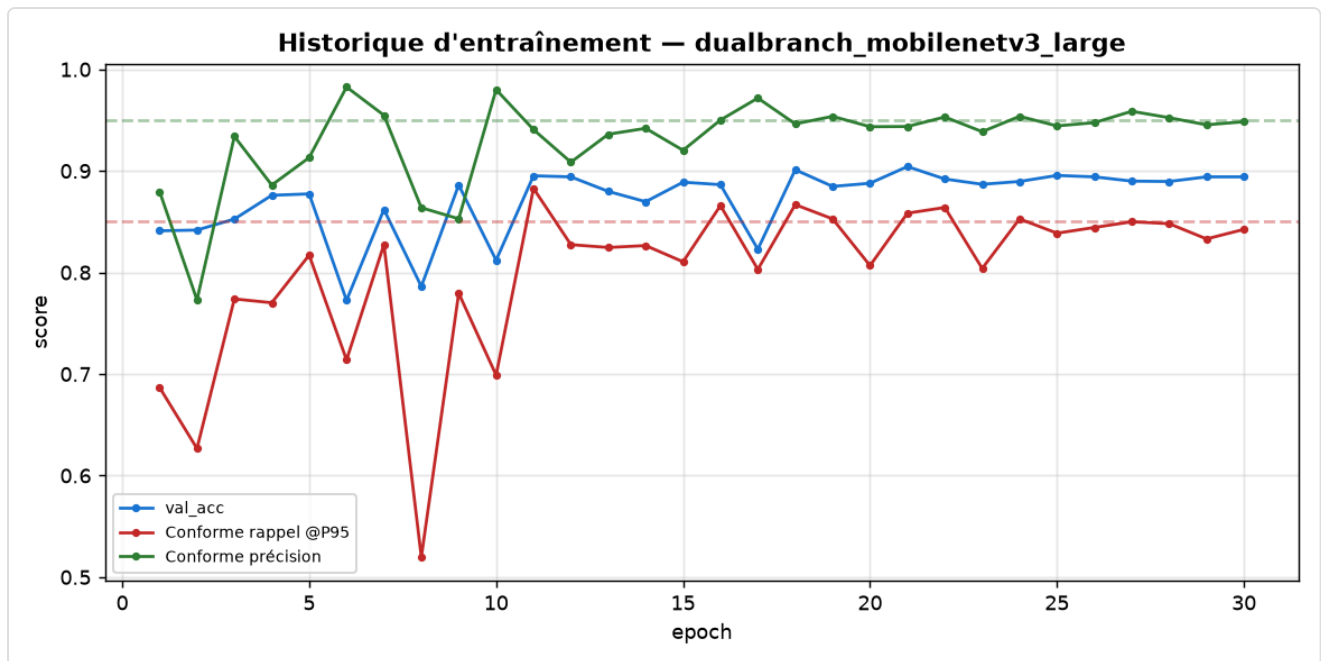
Matrice de confusion — dualbranch (test)				
Vrai	Conforme	NON Conforme	PIETRA	Vide
	963	11	91	2
	8	449	81	3
	57	32	542	7
	1	1	4	610
Prédit				

Tableau des erreurs de la version retenue : les cases de la diagonale sont les bonnes réponses, les autres les confusions.

Les erreurs qui restent sont surtout des confusions entre « bon » et « PIETRA » (des défauts de surface très fins). Le plus rassurant pour la coopérative : **les erreurs vont dans le bon sens** — le logiciel écarte parfois un bon fruit par excès de prudence, mais laisse très rarement passer un mauvais. C'est exactement ce que demande la consigne.

4.5 Vérifier que le résultat est solide

Pour être sûr que ces résultats ne sont pas un coup de chance, j'ai **refait l'expérience 5 fois** sur 5 découpages différents des données. Les résultats restent stables (précision autour de 95 %, rappel autour de 87 %, avec très peu de variation). Le logiciel est donc fiable, pas chanceux.



Évolution des performances au fil de l'apprentissage : ça tâtonne au début, puis ça se stabilise au-dessus des objectifs.

5. Faire tourner le logiciel sur la vraie machine

J'ai converti le logiciel dans un format standard et léger (appelé ONNX), et vérifié qu'il donne exactement les mêmes réponses qu'avant conversion. Son coût est très faible :

Élément	Valeur
Taille du logiciel	~15 Mo
Mémoire utilisée (au repos / en marche)	~75 Mo / ~220 Mo
Temps de réponse pour un cycle de la machine	~0,05 seconde

La carte graphique de la machine dispose de 3 Go de mémoire : on en utilise à peine un dixième, il y a donc une marge très confortable. Le temps de réponse est largement suffisant pour suivre la cadence de tri. En résumé, **toutes les exigences sont respectées**, et la mémoire est le point le plus tranquille. À l'usage, il suffit de donner au logiciel les deux photos d'un fruit pour obtenir sa catégorie.

6. Ce que j'ai utilisé

Outils. Le langage Python ; des bibliothèques spécialisées pour créer et entraîner le logiciel de reconnaissance (PyTorch, MobileNet) ; un outil de traitement d'images (OpenCV) pour découper les vidéos et préparer les photos ; un outil pour découper proprement les données (scikit-learn) ; un carnet de bord automatique des essais (MLflow) ; un format standard pour la mise en production (ONNX) ; l'outil de stockage des images fourni (DVC) et l'outil de vérification des étiquettes (Streamlit) ; et Git pour archiver tout le code.

Ce qui m'a inspiré. Je me suis appuyé sur des projets publics reconnus de tri de fruits et de graines par caméra, qui utilisent la même idée de « plusieurs photos par objet », pour confirmer ma direction.

7. Conclusion

Le logiciel final **respecte la consigne avec de la marge** (95 % de précision et 92 % de rappel sur les fruits jamais vus), tient largement sur la machine de la coopérative, et son résultat est prouvé stable. L'essentiel du travail n'a pas été d'empiler des calculs, mais d'avoir **bien compris le problème** : une châtaigne se juge sur ses deux faces, un défaut d'un seul côté se repère par la différence entre les photos, et dérouler le disque de l'image aide à mieux voir. En gardant toujours la consigne de la coopérative en tête — la sûreté avant tout — j'ai évité de courir après un beau chiffre qui n'aurait pas tenu en vrai.

Pistes pour la suite. Vérifier si une partie des confusions « bon / PIETRA » vient d'erreurs d'étiquetage dans les données ; mesurer le temps de réponse réel sur la machine de la coopérative ; et, si besoin d'un tout petit gain, combiner plusieurs versions du logiciel.